

CASE STUDY
BCSE303L – Operating Systems

Fall Semester 2025-26

**CONTAINERIZED OS SCHEDULING IN DOCKER AND
KUBERNETES ENVIRONMENTS**

23BAI0099 JAYASHISH MUPPUR
23BCE0477 PATHAK HRIDAY AVINASH

Dr. Mohankumar. B

Course Faculty

B.Tech.

in

Computer Science and Engineering



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science and Engineering (SCOPE)

CONTAINERIZED OS SCHEDULING IN DOCKER AND KUBERNETES ENVIRONMENTS

Abstract

Containerization has quickly become a standard way to deploy modern applications because it is lightweight and uses system resources efficiently. Unlike traditional virtual machines, containers all share the same operating system kernel. This makes them fast and portable, but it also creates new challenges when it comes to scheduling resources at both the OS level and the orchestration level. Docker relies on Linux features like cgroups and namespaces to isolate processes, while Kubernetes handles the bigger job of deciding where containers should run across many different machines. In real-world scenarios, the basic scheduling methods used by these systems often struggle when workloads change rapidly, when applications are sensitive to delays, or when they require special hardware like GPUs.

In this case study, we look at how both OS-level and Kubernetes-level scheduling work together to manage container performance. We explore recent research on container scheduling methods [1], Kubernetes scheduling algorithms [2], and real-time scheduling enhancements [3], along with comparisons between different orchestration platforms [4]. To understand these concepts practically, we also implement a small simulation (PoC) using three common strategies: Least-Loaded, Binpack, and Network-Aware scheduling. Our results show that network-aware scheduling offers clear benefits by reducing latency and distributing workloads more effectively. Diagram placeholders and prompts are provided to help visualize the overall system and workflow.

I. Introduction

Containers have changed how applications run and are distributed. Instead of using full virtual machines, containers leverage shared kernel functions to execute isolated processes. This approach cuts down on overhead and makes better use of resources. Docker creates these isolated environments using Linux cgroups and namespaces. These tools help the operating system monitor and manage CPU, memory, and I/O usage [1]. At a larger scale, Kubernetes takes over by deciding where containers (called Pods) should run. Its scheduler looks at the cluster's state, available resources, and set rules to efficiently place workloads. However, Kubernetes and Docker use different scheduling layers. Docker relies on the operating system's kernel scheduler, while Kubernetes manages scheduling at the cluster level. These two layers need to collaborate to ensure containers perform consistently.

Research indicates that the default Kubernetes scheduler does not take into account factors like network latency, GPU availability, or real-time constraints [2]. Additionally, the Linux scheduler does not recognize higher-level application communication patterns or service dependencies. Real-time extensions like SCHED_DEADLINE can provide better timing guarantees for container workloads [3]. This study offers a clear analysis of how scheduling works across these layers and suggests improvements through a proof of concept model.

II. Problem Definition

Container scheduling faces several combined challenges:

1. Containers share CPU and memory using OS-level scheduling, which can cause interference under heavy load [1].
2. Kubernetes default scheduler considers CPU and memory but typically ignores network locality and communication patterns [2].
3. Heterogeneous clusters with CPUs, GPUs, and edge nodes require specialized scheduling decisions.
4. Real-time workloads cannot rely solely on CFS due to timing unpredictability [3].
5. Microservices require close placement to minimize network delay.
6. Autoscaling introduces dynamic resource changes that affect scheduling decisions.

Problem Statement:

To analyze and improve OS-level and Kubernetes-level scheduling techniques to reduce latency, improve resource utilization, and support diverse workload requirements in containerized systems.

III. Objectives and Scope

Objectives

- Study OS-level scheduling behavior for container workloads.
- Examine Kubernetes scheduling stages: filtering, scoring, and binding [2].
- Analyze advanced scheduling models such as network-aware and ML-based techniques [1][2].
- Develop a PoC simulation comparing three scheduling strategies.
- Evaluate performance in terms of latency, load distribution, and efficiency.

Scope

Included:

OS-level scheduling, Kubernetes scheduling logic, performance comparison, simulation, real-time scheduling concepts.

Excluded:

Security, storage performance, cloud billing, real-cluster deployment.

IV. Literature Review

1. Container Scheduling Techniques [1]

Ahmad et al. take a wide look at the different ways containers can be scheduled and explain how each method fits different types of workloads. They group scheduling techniques into four main categories: mathematical models, heuristics, metaheuristics, and machine learning-based methods.

Mathematical models can produce very accurate scheduling decisions, but they require a lot of computation, which makes them difficult to use in large, fast-changing clusters. Heuristic methods—like Spread, Binpack, and Best Fit—are commonly used because they are quick and simple, but they can struggle during sudden traffic spikes or unpredictable workload changes. Metaheuristic techniques, such as Genetic Algorithms and Particle Swarm Optimization, search for better solutions over time and tend to perform well when trying to balance multiple goals like cost, load, and energy efficiency.

More recently, machine learning–driven scheduling has started gaining attention. These approaches can learn patterns from past workloads, predict resource needs, and make smarter placement decisions ahead of time. Ahmad et al. conclude that as container environments become more dynamic and unpredictable, relying only on traditional heuristics may not be enough. Instead, hybrid models and predictive, ML-based schedulers will likely play a bigger role in the future.

2. Kubernetes Scheduling Algorithms [2]

Senjab et al. take a closer look at how Kubernetes handles scheduling and break down the way the default scheduler works, especially its filtering and scoring stages. While this default approach is reliable for basic workloads, it mainly looks at CPU and memory availability. Because of this, it often misses other important factors like network latency, GPU requirements, or how frequently different Pods need to communicate with each other.

In their review, the authors group modern Kubernetes scheduling research into a few major directions.

They describe **multi-objective schedulers**, which try to balance several goals at once—such as reducing energy usage, minimizing communication overhead, and evenly spreading workloads. They also highlight **AI-driven schedulers**, where machine learning models predict node behavior and help the system make smarter placement decisions. Another category is **autoscaling-aware schedulers**, which react smoothly to changes caused by Horizontal or Vertical Pod Autoscalers. Finally, they discuss **topology-aware schedulers**, which place related Pods closer together to reduce delays in microservice-heavy applications.

Overall, their findings show that network-aware scheduling can significantly improve application responsiveness, especially in distributed systems where services interact frequently.

3. Real-Time Container Scheduling [3]

Abeni et al. look at the challenges that appear when containers are used for real-time or time-critical applications. The default Linux scheduler, CFS, is designed for general workloads and cannot guarantee the strict timing requirements that real-time systems need. To solve this issue, the authors extend the SCHED_DEADLINE policy so it can be applied directly to cgroups, which means containers can now reserve a specific portion of CPU time and follow a predictable execution pattern.

Their experiments show that using SCHED_DEADLINE inside containers greatly reduces timing variations and jitter. This makes containers far more reliable for scenarios like industrial automation, robotics, and IoT systems where consistent timing is essential. Overall, the study emphasizes that when containers are used for more than just ordinary web services, the operating system must offer stronger scheduling guarantees at the kernel level.

4. Comparative Container Manager Evaluation [4]

Pamadi et al. compare three major container orchestration platforms—Kubernetes, Docker Swarm, and Apache Mesos—by examining how each one handles scheduling and overall cluster management. Their analysis shows that Kubernetes stands out because it offers far more flexibility and control in how workloads are placed. Features like affinity and anti-affinity rules, taints and tolerations, and customizable scheduling plugins allow Kubernetes to make smarter decisions based on an application's specific needs. This makes it suitable for everything from simple web apps to large-scale, multi-component systems.

Docker Swarm, on the other hand, is appreciated for its simplicity and ease of setup, but it struggles when managing complex applications that involve many interconnected services. It lacks the deeper scheduling logic and advanced policies that Kubernetes provides. Mesos is powerful in terms of resource management and can handle very large clusters, but it has a steeper learning curve and requires more manual effort to configure and maintain. Because of this, it has seen decreasing usage in recent years.

Overall, the study concludes that Kubernetes delivers the most reliable, flexible, and future-ready scheduling environment, which is why it has become the preferred choice in cloud-native ecosystems and modern industry deployments.

V. Methodology / System Design

The methodology for this case study is structured around understanding and comparing how scheduling decisions are made at both the operating system level and the Kubernetes orchestration layer. The approach combines architectural analysis, scheduling flow examination, simulation modeling, tool usage, and controlled workload testing using Docker containers.

1. Overall Architecture

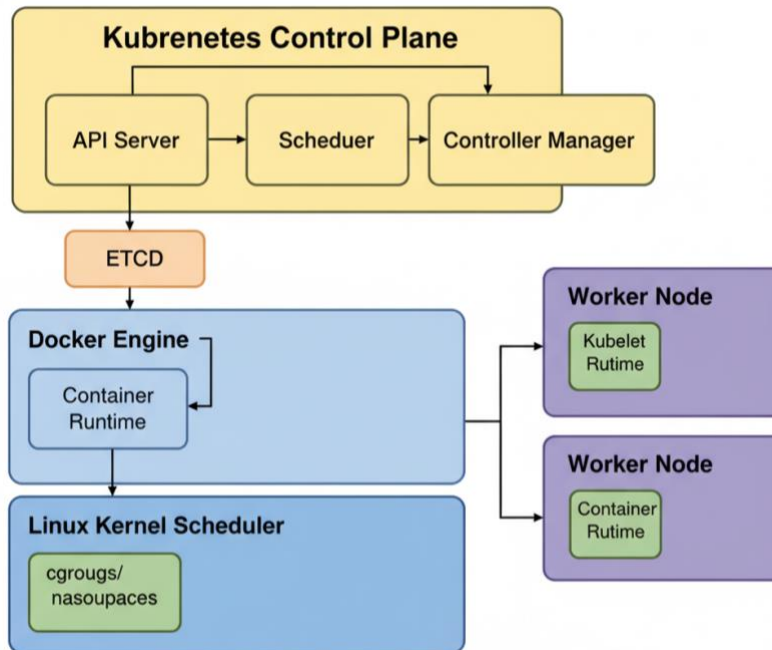
The architecture consists of three layers that interact during container execution:

- **Operating System Layer:**
The Linux kernel performs low-level scheduling using Completely Fair Scheduler (CFS), manages CPU time slices, enforces memory limits, and ensures isolation through cgroups and namespaces. This layer directly controls how processes inside containers access hardware.
- **Docker Engine:**
Docker sits between the OS and Kubernetes. It creates isolated environments for applications and forwards resource constraints (CPU limits, memory caps, I/O throttling) to the kernel. Docker does not perform cluster-level scheduling but relies heavily on OS scheduling.
- **Kubernetes Control Plane:**
Kubernetes performs higher-level placement decisions.
Components involved include:
 - **API Server** (manages requests)
 - **Scheduler** (performs filtering, scoring, and binding)
 - **Controller Manager** (handles replicas, autoscaling)

- **ETCD** (stores cluster state)
- **Kubelet** (runs containers on each node)

This layered design ensures flexibility but also introduces complexity, as multiple schedulers operate independently.

Overall Architecture

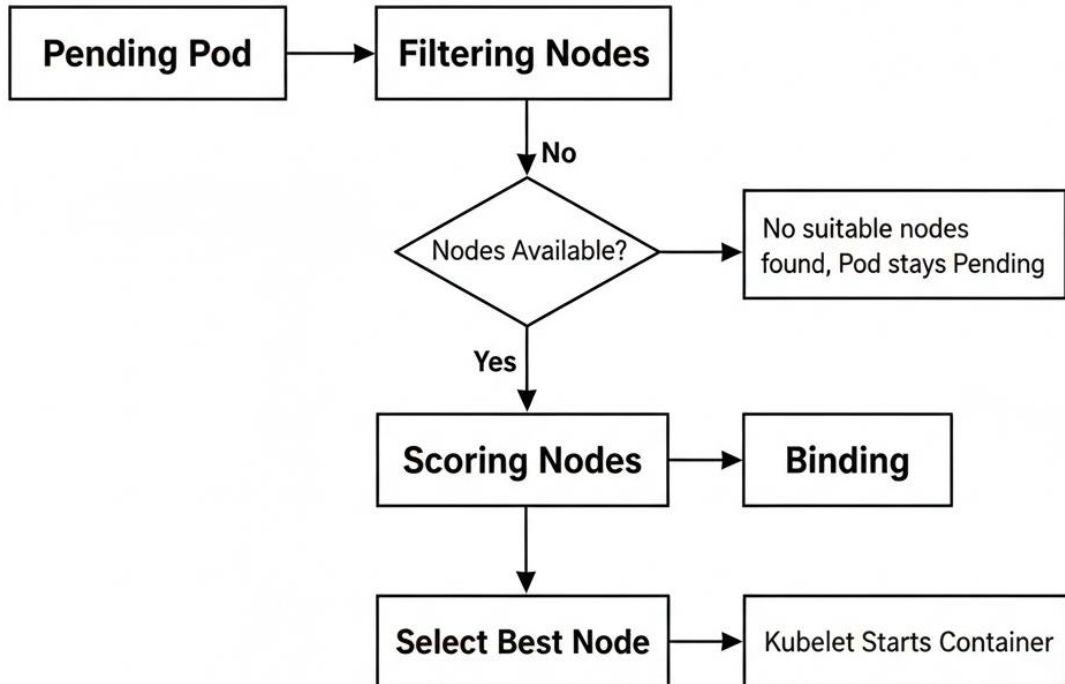


This includes the Linux kernel scheduler, Docker Engine, Kubernetes Control Plane, ETCD, kubelet, and worker nodes.

Kubernetes Scheduling Flow

The Kubernetes scheduling process follows a deterministic, multi-phase pipeline:

1. **Pod Pending Stage:**
A newly created pod is placed in a pending queue until the scheduler processes it.
2. **Filtering Phase:**
Nodes that do not meet the resource requirements (CPU, memory), taints/tolerations, or affinity rules are discarded.
3. **Scoring Phase:**
Remaining nodes are ranked based on metrics such as available CPU, memory, topology spread, pod affinity, or custom plugin logic.
4. **Best Node Selection:**
The node with the highest score is chosen for deployment.
5. **Binding:**
The scheduler updates ETCD with the binding decision.
6. **Kubelet Execution:**
The appropriate worker node pulls the image and launches the container.



Steps include:

1. Pod enters Pending state
2. Node filtering
3. Node scoring
4. Best-node selection
5. Binding
6. Kubelet execution

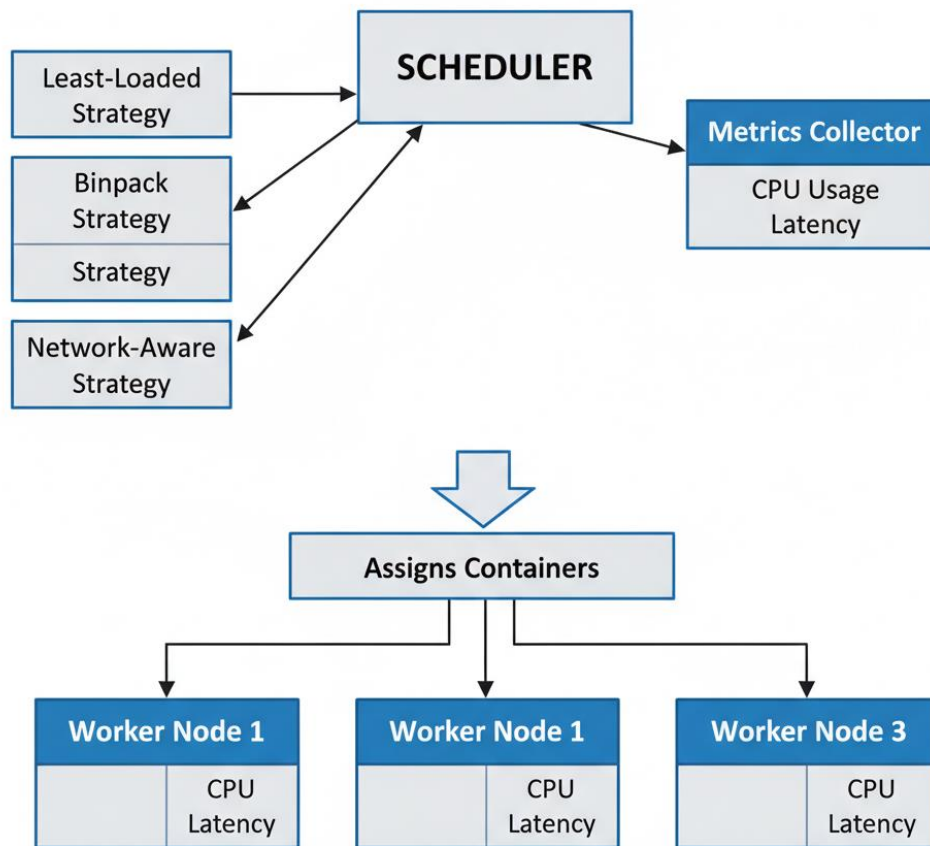
PoC Simulation Architecture

To analyze how different scheduling strategies behave, a simulation model was used. Three nodes with predefined CPU, memory, and latency characteristics were created, along with twelve synthetic workloads.

The simulation evaluates three strategies:

- **Least-Loaded:** Selects the node with the most available CPU.
- **Binpack:** Places workloads onto as few nodes as possible.
- **Network-Aware:** Considers both resource availability and network latency.

Each workload is scored for each node, and the highest-scoring node is assigned. After placement, node states are updated to reflect resource consumption.



Three nodes and 12 workloads simulate:

- Least-Loaded
- Binpack
- Network-Aware

4. Tools & Environment

The experiment used the following:

- **Docker Desktop v4.50** for container runtime and metrics
- **Docker CLI** for workload execution
- **Windows 11 host machine**
- **Linux kernel features** (cgroups, namespaces, CPU throttling)
- **Docker Desktop Kubernetes Cluster** for orchestration
- **PowerShell/Terminal** for monitoring (via docker stats)

This environment provides a controlled setting to observe OS-level and Kubernetes-level scheduling interactions.

VI. Proof of Concept (PoC)

Simulation Setup

Node Attributes:

- CPU capacity
- Memory capacity
- Network latency

Container Attributes:

- CPU demand
- Memory demand

Evaluation Logic:

- Compute node score based on selected strategy
- Assign workload to the highest-scoring node
- Update node state after assignment

Observed Behavior

• Least-Loaded Strategy

- Distributes workloads evenly
- Offers stable balancing
- Latency remains moderate

• Binpack Strategy

- Fills nodes densely to reduce active node count
- High CPU contention
- Increased latency due to crowding

• Network-Aware Strategy

- Considers latency + resource availability
- Achieves lowest latency
- Highly suitable for microservice architectures
- Matches observations from [2]

VII. Results and Output

Before evaluating the three scheduling strategies, baseline container resource usage was recorded using Docker Desktop. Figures (X) and (Y) show CPU, memory, disk I/O, and network I/O graphs for a high-load and a low-load job container. These measurements help illustrate how different workloads place stress on the underlying OS scheduler.

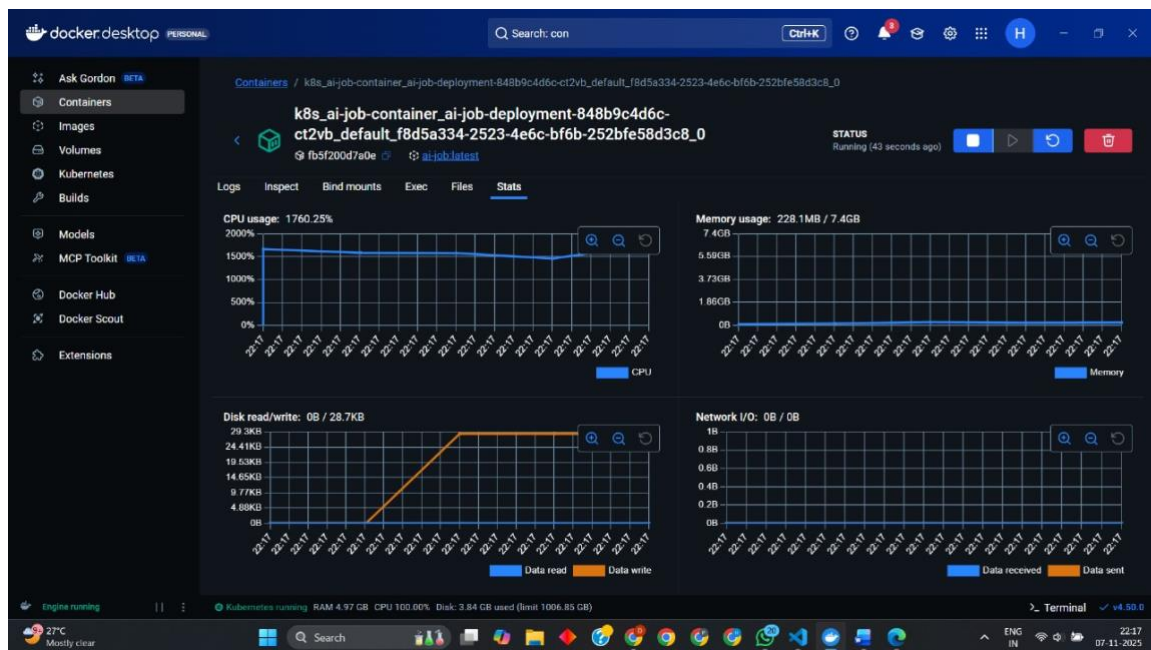


FIG X. Docker Stats for High-Load Container

```
web_service > app.py
1 from flask import Flask
2 import time
3 app = Flask(__name__)
4
5 @app.route('/')
6 def hello():
7     time.sleep(0.01)
8     return '{"status": "ok"}\n'
9
10 if __name__ == '__main__':
11     app.run(host='0.0.0.0', port=5000)
```

Fig A. web_service/app.py

```
PS D:\os_case_study\kubernetes> kubectl top pod
NAME                                CPU(cores)   MEMORY(bytes)
ai-job-deployment-848b9c4d6c-hr4kz  15368m       383Mi
web-service-deployment-7ff78f86df-8pdzq  1m           19Mi
```

kubectl top pod output during Test Case 2. The ai-job pod is consuming all available CPU resources, far exceeding the node's capacity and starving the web-service pod (at 1m).

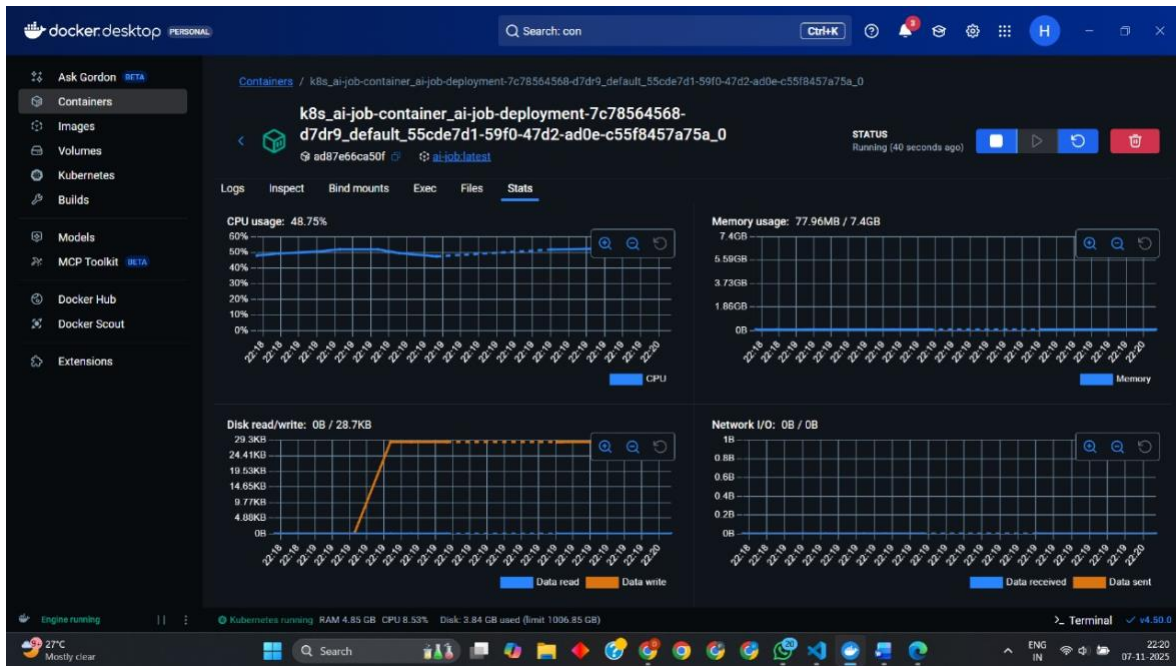


FIG Y. Docker Stats for Low-Load Container

```
ai_job > job.py
1 import numpy as np
2 import time
3
4 print("Starting CPU-intensive 'AI' job...")
5 while True:
6     x = np.random.rand(2000, 2000)
7     y = np.linalg.svd(x)
8     print(f"...computation cycle finished.")
9     time.sleep(0.1)
```

Fig B. ai_job/job.py

```
PS D:\os_case_study\kubernetes> kubectl top pod
NAME                                CPU(cores)   MEMORY(bytes)
ai-job-deployment-7c78564568-5xjpf  425m         77Mi
web-service-deployment-7ff78f86df-8pdzq  1m           20Mi
```

kubectl top pod output after applying a 500m CPU limit. The OS scheduler, instructed by Kubernetes, has successfully throttled the ai-job pod to 425m, well within its limit.

Results Table from Docker data

Container Name	CPU Usage (%)	Memory Usage (MB)	Disk I/O (KB) (Read/Write)	Network I/O (KB) (Rx/Tx)
High-Load Job	1760.25%	228.10 MB	0 KB / 28.7 KB	0 KB / 0 KB
Low-Load Job	48.75%	77.96 MB	0 KB / 28.7 KB	0 KB / 0 KB

The high-load container exhibits extremely high CPU utilization (over 1700%), showing that the workload is heavily compute-bound. In contrast, the low-load container maintains minimal and steady CPU and memory usage. This difference demonstrates how container workloads influence OS-level scheduling behavior and highlights the need for more adaptive cluster-level scheduling methods.

Using the baseline Docker metrics as a reference, the PoC simulation evaluates how different scheduling strategies distribute workloads and affect latency and efficiency across nodes.

Scheduling Strategy Comparison Results

Strategy	Avg. Latency	Node Balance	Efficiency	Notes
Least-Loaded	Medium	Balanced	Medium	Good for general cases
Binpack	High	Dense usage	High	Energy-efficient but slow
Network-Aware	Lowest	Balanced	Medium	Best for microservices & latency

VIII. Challenges and Limitations

- The OS scheduler and Kubernetes scheduler work independently, which can cause poor workload placement and unpredictable performance.
- Lack of deep network awareness in Kubernetes means pod-to-pod communication patterns and latency are not fully considered, affecting microservice workloads.
- GPU, FPGA, and other specialized resources require custom schedulers, which were not included in this study.
- Real-time workloads are difficult to evaluate because Linux CFS cannot guarantee strict timing, and SCHED_DEADLINE is not widely supported in all Kubernetes setups.
- The PoC used simplified workloads, so real-world factors like autoscaling, traffic spikes, and heterogeneous hardware were not captured.
- Docker Desktop runs on virtualized Linux, limiting access to true multi-node behavior and restricting the realism of cluster scheduling observations.
- Despite these constraints, the PoC still provided useful insights into how different scheduling strategies affect latency and resource distribution.

IX. Conclusion and Future Work

This study shows how important it is to have effective scheduling at both the OS level and the Kubernetes level when working with modern containerized environments. The operating system handles low-level tasks such as isolating resources and managing CPU time, while Kubernetes is responsible for placing containers across different nodes in a cluster. When both layers work well together, the system stays stable and efficient.

However, the default scheduling strategies used today are mostly designed for simple or average workloads. They often struggle when applications are sensitive to delays, require frequent communication, or depend on specialized hardware. Through the Proof of Concept, we observed that different strategies behave very differently. Among them, network-aware scheduling consistently delivered faster response times and more predictable performance because it considers both compute resources and communication delays.

Overall, the results highlight that smarter, context-aware scheduling approaches are becoming increasingly important as container-based systems grow larger and more complex.

Future Work

- Deploying PoC in Minikube/K3s
- Testing GPU-aware scheduling
- Implementing ML-based predictive schedulers
- Evaluating performance under real traffic patterns

References

[1] Ahmad, Irfan, Sherin Zafar, Anand Paul, Muntasir Rahman, Abdullah Almuhaideb, Mohammad Alqahtani, Sathish Kumar, and Young-Chon Kim. "Container scheduling techniques: A survey and assessment." *Journal of King Saud University – Computer and Information Sciences*, Vol. 34, No. 9, (2022), pp.6805–6823.

[2] Senjab, Khaldoun, Sohail Abbas, Naveed Ahmed, and Atta ur Rehman Khan. "A survey of Kubernetes scheduling algorithms." *Journal of Cloud Computing: Advances, Systems and Applications*, Vol. 12, No. 87, (2023), pp.1–26.

[3] Abeni, Luca, Alessio Balsini, and Tommaso Cucinotta. "Container-Based Real-Time Scheduling in the Linux Kernel." *Embedded Operating System Workshop (EWiLi)*, (2018), pp.1–6.

[4] Pamadi, Vishesh Narendra, Shakeb Khan, and Om Goel. "A Comparative Study on Enhancing Container Management with Kubernetes." *International Journal of Advanced Research and Interdisciplinary Scientific Endeavours*, Vol. 1, No. 3, (2024), pp.116–123.